Lecture 22.07: Differentiation Rules & Activation Function Derivatives

📌 Topic Overview

In this lecture, Shraddha ma'am details the fundamental **Rules of Differentiation** (Power Rule, Product Rule, Quotient Rule, Chain Rule) and applies them to calculate derivatives of key neural network **Activation Functions**.

🏛️ Essential Differentiation Rules

Rule Name	Mathematical Formulation	Example
Power Rule	$\frac{d}{dx}[x^n] = nx^{n-1}$	$\frac{d}{dx}[x^3] = 3x^2$
Constant Multiple	$\frac{d}{dx}[cf(x)] = cf'(x)$	$\frac{d}{dx}[5x^2] = 10x$
Sum / Difference	$\frac{d}{dx}[f \pm g] = f' \pm g'$	$\frac{d}{dx}[x^2 + 3x] = 2x + 3$
Product Rule	$\frac{d}{dx}[f \cdot g] = f'g + fg'$	$\frac{d}{dx}[xe^x] = e^x + xe^x$
Quotient Rule	$\frac{d}{dx}\left[\frac{f}{g}\right] = \frac{f'g - fg'}{g^2}$	$\frac{d}{dx}\left[\frac{x}{x+1}\right] = \frac{1}{(x+1)^2}$
Chain Rule	$\frac{d}{dx}[g(f(x))] = g'(f(x)) \cdot f'(x)$	$\frac{d}{dx}[(2x+1)^3] = 3(2x+1)^2 \cdot 2$

🧠 Activation Function Derivatives

1. Sigmoid Function: $\sigma(z) = \frac{1}{1+e^{-z}}$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

At $z = 0$, $\sigma'(0) = 0.5 \times 0.5 = 0.25$. As $|z| \to \infty$, derivative vanishes to 0 (Vanishing Gradient Problem).

2. ReLU Function: $f(z) = \max(0, z)$

$$f'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Derivative is either 1 or 0, preventing vanishing gradients during backpropagation.

💻 Python Implementation

```
import numpy as np

def differentiate_activation_functions(z_vals=None):
    """
    Computes analytical derivatives of common neural network activation functions:
    1. Sigmoid: sigma(z) = 1 / (1 + e^-z)
       Derivative: sigma'(z) = sigma(z) * (1 - sigma(z))
    2. ReLU: f(z) = max(0, z)
       Derivative: f'(z) = 1 if z > 0 else 0
    3. Tanh: tanh(z)
       Derivative: tanh'(z) = 1 - tanh^2(z)
    """
    if z_vals is None:
        z_vals = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])

    # 1. Sigmoid
    sig = 1.0 / (1.0 + np.exp(-z_vals))
    sig_deriv = sig * (1.0 - sig)

    # 2. ReLU
    relu = np.maximum(0.0, z_vals)
    relu_deriv = np.where(z_vals > 0, 1.0, 0.0)

    # 3. Tanh
    tanh_val = np.tanh(z_vals)
    tanh_deriv = 1.0 - tanh_val ** 2

    return {
        'z': z_vals,
        'sigmoid_deriv': sig_deriv,
        'relu_deriv': relu_deriv,
        'tanh_deriv': tanh_deriv
    }

# --- Function Call & Testing ---
activations = differentiate_activation_functions()
print(f"{'Input z':<8} | {'Sigmoid Deriv':<15} | {'ReLU Deriv':<12} | {'Tanh Deriv
print("-" * 55)
```

```
for i in range(len(activations['z'])):
    print(f"{activations['z'][i]:<8.1f} | {activations['sigmoid_deriv'][i]:<15.4f}
```

🔑 Key Takeaways

1. The power rule and chain rule form the mathematical bedrock of backpropagation.
2. Sigmoid's maximum derivative is only 0.25, which causes vanishing gradients in deep networks.
3. ReLU maintains a gradient of 1.0 for positive inputs, solving the vanishing gradient issue.